

Game Design Document — ESports Simulator

Working title: ESports Simulator (repo: [esports-tycoon](#)) **Genre:** Esports management sim / tycoon, tick-level tactical shooter sim underneath **Reference point:** *Esports Manager 2026* (Steam #2749950), Valorant (setting/flavor), Football Manager (management-depth ambition) **Status:** Playable, multi-season, browser + terminal. Actively in development. **Last updated:** 2026-07-07

1. Vision

You run a professional Valorant-flavored esports organization. You don't aim, peek, or spray — your players do that, and how well they do it is a function of who you signed, how you trained them, how tired and happy they are, what utility they popped, and where they were standing when the fight started. Your job is everything around the ten minutes of a round: scouting and signing talent, building a training program, managing morale and burnout, calling in a coach, negotiating contracts, chasing sponsors, picking (and banning) maps, and reading the story the season tells through its results.

The foundational bet, stated plainly: **a deterministic simulation can feel alive**. Two matches between the same two rosters, seeded differently, should read as genuinely different stories — not because the engine rolls noise on top of a fixed outcome, but because who was standing where, who peeked first, who popped a smoke a half-second late, and who was on a three-game losing streak all compound into a different match. Nothing in the sim is theater; every number the manager sees traces back to an event in the log, and every event traces back to a player attribute, a map feature, or a decision the manager made.

Design pillars

1. **The sim is the truth, not a curtain.** No hidden dice roll decides a match and then generates flavor text to match. Positions, ranges, cover, line-of-sight, and attributes produce the outcome; the outcome is what gets narrated.
 2. **Determinism is a feature, not an implementation detail.** Same seed, same decisions → byte-identical replay, forever. This is what makes "why did we lose that round" answerable, what makes an LLM or an RL agent a viable player, and what will eventually make a world model trainable on real play data.
 3. **You manage humans, not spreadsheets.** Morale, tilt, contract anxiety, and personality clashes are first-class systems, not flavor text bolted onto a stat block.
 4. **Depth without pixels.** The tactical layer doesn't need a game engine to feel real — a floor plan, real distances, cover, and elevation get you 90% of the tactical richness of a full 3D shooter at a fraction of the complexity, and it stays fast enough to sim a whole season in seconds.
 5. **Everything is data until proven otherwise.** Agents, weapons, maps, and their floor geometry are YAML. A new agent or map is a content change, not a code change.
-

2. The core loop



Above this loop sits the **season loop** (regular season → playoffs → offseason → next season, indefinitely) and beneath it sits the **match loop**, which the manager mostly *watches* rather than plays — an entire tactical shooter's worth of simulation resolves in milliseconds, then gets replayed at whatever pace the manager wants.

3. The management layer

3.1 Your organization

You pick one team from the league to manage (two hand-authored "starter" orgs exist — Team Nexus and Team Vanguard — plus generated league teams). Your org has a roster (5 active players), a balance sheet, reputation, fan count, world rank, and team chemistry.

3.2 Players

Every player is a bundle of ten numeric attributes across five categories, scaled 1–99 (Football-Manager-style, not a 0–1 float, because humans read that scale better):

Category	Attributes
----------	------------

Mechanical	Aim Precision, Aim Reactivity, Movement
Tactical	Game Sense, Utility Usage, Positioning
Mental	Clutch Factor, Tilt Resistance, Composure
Team	Comms Quality

Attributes are a registry (`data/attributes.yaml`), not a hardcoded struct — adding an eleventh attribute is a data change. On top of attributes, every player carries:

- **Role** (Duelist / Controller / Initiator / Sentinel / Flex) and **Playstyle** (IGL, Entry, Anchor, Lurker, AWPer, Support) — orthogonal axes that drive both squad-building decisions and in-match behavior.
- **Agent pool & map pool** — per-agent and per-map mastery, so a player's third-best agent is meaningfully worse than their main.
- **Career state** — salary, contract length, morale, stamina, form, age, and free-form personality tags (`hot_head`, `veteran`, `rookie`, `star_player`, `volatile`, ...) that modulate how they respond to talks, tilt, and pressure.
- **Condition** — morale/stamina/form move week to week from training, match results, and rest, and they feed back into match performance (a burnt-out star plays worse than their attributes suggest).

3.3 Training

Each week you set one focus — mechanical, tactical, mental, team, or rest — for your whole roster (the AI picks independently for its own teams, informed by roster needs). Growth follows age curves (young players improve faster, veterans decay), and rest recovers stamina at the cost of growth. A hired **coach** multiplies training gains.

3.4 Transfer market & contracts

A pool of free agents (deterministically generated, refreshed every offseason) is available to sign. Players ask for salaries scaled to their quality; signing, releasing (with a severance cost), and renewing are all manager actions. Contracts run down week by week; a player inside ~8 weeks of free agency with good form will press you for a renewal conversation — ignore it and morale suffers. AI teams work the same market against you.

3.5 Scouting

Rival rosters aren't shown at full fidelity — attributes render with a noise band (`visibility/scouting_uncertainty`, reserved from day one for exactly this). Assign your scout to a target team; the fog shrinks over roughly three weeks of dedicated scouting, faster with a hired **analyst**. The report resets every offseason as rosters change.

3.6 Staff

Three backroom hires, each multiplying one system: a **coach** (training growth), an **analyst** (scouting speed), a **physio** (weekly stamina recovery). A small candidate market regenerates deterministically every season; hiring is instant, releasing is free.

3.7 The Talk module

Once a week you can sit down with one of your players. The topic isn't picked by the manager — it's read off that player's actual state, in priority order: **low morale** → **expiring contract** → **low stamina** → **poor form** → a generic check-in if nothing's actually wrong. You pick one of three approaches (reassure / challenge / listen, or the topic-appropriate equivalent), and the outcome is modulated by that player's personality tags with a deterministic roll — a **hot_head** bristles at being challenged more often than a **calm veteran** does. Effects are small on purpose: a talk is a nudge, not a lever you crank.

3.8 Finances

Weekly income (sponsorships scaled by reputation and fan count, plus prize money) against weekly expenses (payroll, facilities). **Sponsorship deals** arrive as time-limited offers — upfront, steady, or performance-scaled payout structures — that you accept or decline. There's no hard bankruptcy wall yet, but the numbers are real and a mismanaged org can dig a genuine hole.

3.9 Season structure

An 8-team league runs a double round-robin regular season (14–18 weeks depending on team count), then a BO3 playoff bracket with **map veto** (mastery-driven ban/ban/pick/pick/remaining over the 5-map pool) down to a champion, then an offseason (aging, awards, roster churn, free-agent pool refresh) before the next season begins. Campaigns run indefinitely — there is no scripted ending.

3.10 Analytics & storylines

Season-long stat aggregation (K/D/A, an HLTV-flavored rating, first kills, trade kills, headshot %, plants/defuses per player; attack/defense round-win % and pistol conversion per team) feeds a league-leaders board, a team-tendencies view, and **season awards** (MVP, Top Fragger, Opening King, Rookie of the Season) handed out at season end.

News isn't generic — recaps are **templated and grounded**: every fact in a recap sentence resolves to a real event in that match's log (a **head_to_head** helper tracks in-season streaks, revenge results, and "beat the reigning champions" storylines, and cites them only when genuinely notable — silence beats invented drama). Phrasing is seeded per event so the same result always reads the same way, but different results read differently, in a dry, understated, no-hype voice (see [docs/salvage/tone_and_cast_lock.md](#) for the style bible this follows).

4. The match simulation

This is the part of the game the manager mostly *watches*, but it's where almost all of the engineering lives, because it's the thing that has to convincingly justify every result the management layer reports.

4.1 What a match is

A best-of-1 (or, in playoffs, one map of a BO3) plays out as **first to 13 rounds, halftime side swap at 12, overtime if tied 12–12** (win by 2, capped). Each round is simulated tick by tick (1 tick = 0.5 game-seconds) through: a buy phase, live round play, post-plant (if the spike goes down), and round end. A full match resolves in roughly 50 milliseconds and produces a canonical, replayable event log.

4.2 The map: a floor plan, not just a graph

Under the hood, a map is a directed graph of named **callouts** ("A Site", "Mid Courtyard", "B Long") with traversal edges and sightlines — this is the sim's *decision* vocabulary, and it's what keeps the tactical AI and the eventual RL/world-model work legible (an agent reasons in terms of "hold A short" or "rotate to B," not raw pixels).

But every callout also has a **physical room**: an axis-aligned rect on a shared 0–100 grid, with:

- **Corridors** — explicit waypoint paths for connections whose rooms don't directly touch, so a rotate traces an actual hallway instead of teleporting.
- **Props** — half-height crates (cover: you can shoot over them, but they block shots from the far side) and full-height boxes (they break line-of-sight outright — even between two players standing in the same room, like Ascent's mid box).
- **Elevation** — rooms like Haven's A-Heaven or Split's B-Rafters sit above the ground floor and grant a real high-ground bonus to anyone looking down into the site below.

Five maps currently exist, each individually hand-tuned: **Haven** (3-site), **Ascent** (2-site, open mid), **Bind** (2-site, no mid — its identity is the direct site-to-site link, not verticality), **Lotus** (3-site), **Split** (2-site, tall mid spine).

Authoring a map is a YAML content change (`data/maps/<id>.yaml` for the graph, plus `data/maps/geometry/<id>.yaml` for the floor plan) validated by a test suite that guarantees no movement clips through a wall and every room sits where its callout anchor says it should.

Map gimmicks — rotating doors (Lotus), teleporters (Bind), and breakable doors that can start a round shut (Ascent) — exist as a real edge-level mechanic in the schema and engine: every use is *loud*, and enemies within a noise radius react (a watch direction snaps toward the sound; a pre-plant defense can treat it as a rotation trigger), which is also what makes faking a gimmick a legitimate read. This system is implemented but not yet authored onto any specific map's data file — the plumbing exists, the content pass is still open.

4.3 Continuous movement

Players are not graph tokens — every player holds a real (x, y) position every tick. At round start, they don't stack on a room's center; they take a **tactical slot**: a spot behind a specific crate (cover), just inside a doorway (portal, for holding an angle out), or one of several interior spread points, chosen deterministically (hash-spread per player+room, so five teammates never pile onto the same box). Movement between rooms follows a real path — slot → corridor/portal → slot — at a speed scaled by the player's Movement attribute, so a fast player's rotate is genuinely faster, not just flavor text.

Pacing is a designed, measured constant, not an emergent accident: an attacker rotating from one site's approach to another's, through their own spawn, takes roughly 30 seconds — matching real Valorant's rhythm. A defender's equivalent rotate, through their own interior lines, is always meaningfully faster than the attacker's version of the same trip — that speed gap is the defense's structural advantage for having to guess which of several sites gets hit. Every map is measured against this rule by an automated pacing report and re-tuned when it drifts.

4.4 Duels

When two players from opposing teams have a mutual sightline, they may engage. Whether they do, and who wins if they do, is a function of real things:

- **Range.** The fight happens at the actual distance between the two players' positions — not an abstract "room A vs room B." Snipers want long sightlines and are penalized close up; SMGs and pistols are the reverse; rifles are flat everywhere.
- **Cover & elevation.** A player tucked behind the right crate relative to the shooter gets a real bonus; the higher player across an elevation gap gets a real bonus.
- **Line of sight.** A full-height prop between the two exact positions breaks the engagement outright, even mid-room.
- **Facing.** Stationary players pre-aim toward the threat side. That bonus only pays out inside the cone they're actually watching — a shot from outside it is a **flank**, and flanks are punished, not just ignored. Lurking through a cleared angle is a real, rewarded tactic now.
- **Weapon, armor, agent mastery, map mastery**, and all ten player attributes (aim precision/reactivity, movement, positioning vs. game sense depending on who's holding vs. entering, clutch factor in 1vX, tilt resistance on a losing streak, composure).
- **Day form** — correlated per-match noise (a whole team can show up "hot" or "cold" for the match, scaled down by composure), which is why two matches between the same rosters produce genuinely different stories instead of the stronger side grinding out an identical win every time.

4.5 Micro-combat

Below the duel-resolution layer, individual fights have texture:

- **Peeking** — an aggressive player (entries, AWPers) on a stalemated angle will sometimes swing it deliberately, trading their holder status for initiative. Peeks fizzle (both sides bail) more often than a

standard poke — that's the "jiggle-peek" pattern — and a peeker with a flash charge in reserve will sometimes pop it on the way in.

- **Fizzled duels and post-kill repositioning** trigger real micro-movement: a player shuffles a few units to a nearby cover slot, emitted as an actual movement event, so a replay shows the footwork of a fight, not just its resolution.
- **Utility** is individually attributed and consumed: smokes block sightlines for a window, flashes debuff whoever they catch, info abilities (recon darts, drones) can trigger an IGL to re-call a stacked site to the weaker one mid-approach, or let a defending initiator shave time off a rotation call once a round.

4.6 Economy, retakes, and saves

Valorant's real credit rules apply: pistol rounds, win/loss bonuses that scale with consecutive losses, a plant bonus, an armor/weapon economy, and force-buy/eco/full-buy decisions the (AI) IGL makes off the team's average bank. Post-plant, defenders don't feed into a retake one at a time — outnumbered site defenders **fall back** instead of dying in place (breaking contact with a brief disengage grace), rally with rotating teammates, and either mount a **grouped retake** with real numbers or **save** their weapons and concede the round if the situation is hopeless. This asymmetric behavior — not any symmetric tuning knob — is what finally got the attack/defense round-win balance into a realistic band; the design history here is a genuine lesson (see §7).

4.7 Determinism

Every stochastic decision in the sim draws from a hierarchical RNG (`rng/tree.py`) seeded by a label path — (`match, round, player, event`) — derived via keyed hashing from a single root seed. Two runs of the same match, same seed, produce a **byte-identical event log**, guaranteed by a dedicated test (`tests/test_determinism.py`) and reinforced by a **golden-file gate**: a canonical match's log hash is committed to the repo, and any engine change that alters it — intentionally or not — fails CI until the fixture is deliberately re-blessed. This is the load-bearing architectural bet: it's what makes replays trustworthy, what will make an RL agent's or LLM's play reproducible for debugging, and what will eventually make a world model trainable on logged play.

4.8 The event log is the only truth

Every kill, plant, defuse, buy, utility use, movement, round-start, and match-end is a typed Pydantic event (`schemas/events.py`), appended to an ordered, JSONL-persistable log. Nothing downstream — the CLI scoreboard, the web viewer, the season stats, the recap generator — holds independent state. They are all pure readers of the same log. This is enforced as a standing architecture-review rule on every change to the UI layer.

5. Presentation

5.1 The web app

A FastAPI backend (`web/server.py`) exposes the same `GameState` the terminal CLI drives, as JSON views (dashboard, roster, standings, schedule, market, stats, finances) and typed actions (train, sign, release, renew, talk, scout, sponsor respond, hire/release staff, advance week). The frontend is a no-build-step vanilla-JS app on a custom design system (`ui/design-system/` — dark-first, information-dense, a Valorant-red accent used sparingly).

5.2 The match viewer

Any played match can be replayed from a floor-plan **isometric** view (with a 2D top-down toggle): real rooms, extruded walls, tinted sites, corridor walkways, players walking their actual paths with motion trails, kill markers, utility markers (color- and shape-coded by ability type — smoke, flash, damage, info, ultimate), a live kill/utility feed, the round clock (with a post-plant amber state), and full playback control — 1×/4×/16×/ instant speed, pause, scrub, round-skip. The viewer is a pure consumer of the event log; it holds no simulation state of its own, and a legacy-log fallback keeps older replays (pre-geometry, pre-continuous-movement) playable.

5.3 Art

A small, deliberately curated asset pack (generated via the Ludo.ai API, committed once rather than generated on demand): a title splash, eight team logos, and ten role-based player portraits, assigned deterministically by entity-id hash so the same team or role always gets the same art.

5.4 The terminal

The original interface — a `rich`-based CLI (`app/cli.py`) — remains fully supported alongside the web app: `python -m esports_sim` for an interactive session, or `--auto N --seed S --team T` for a fully headless N-week run with no UI at all, which doubles as the sim's load-bearing regression harness.

6. Content roster (current)

Category	Count	Examples
Agents	13	Jett, Raze, Reyna, Phoenix (duelists) · Omen, Viper, Clove (controllers) · Sova, Breach, Skye (initiators) · Killjoy, Cypher, Chamber (sentinels)
Weapons	7	Classic, Ghost, Sheriff (pistols) · Spectre (SMG) · Phantom, Vandal (rifles) · Operator (sniper)
Maps	5	Haven, Ascent, Bind, Lotus, Split — each with an authored floor-plan geometry layer
Attributes	10	Registry-driven; adding an 11th is a data change
Teams	2 starter + generated league	Team Nexus, Team Vanguard, plus a deterministically generated league fill

All of the above are YAML under `data/`. None of it is a Riot Games asset or real player likeness — rosters, orgs, and agent kits are original fictional content in a Valorant-flavored idiom.

7. Design history worth knowing

A few hard-won lessons are encoded in the current tuning and worth preserving so future work doesn't relearn them the expensive way:

- **Symmetric knobs can't fix an asymmetric problem.** Early attack/defense balance sat 63–72% in attackers' favor. Every symmetric lever tried — hold-advantage buffs, utility stalls, raising the pre-commit poke rate — either didn't move it or made it *worse* (raising poke rate favors whichever side has more bodies to spend attacking a 5-vs-2 site, which is the attackers). The fix that actually worked was asymmetric: give outnumbered defenders a real fallback/retake/save behavior instead of feeding them into the crossfire one at a time. That dropped the range to a realistic 45–65% band across all five maps.
- **A single stronger-roster-always-wins simulation isn't "alive."** Correlated per-match "day form" noise (scaled down by composure) was necessary so that two matches between the same two rosters produce

visibly different stories — upsets happen, and they happen for a legible reason (a team ran cold, not a coin flip).

- **Geometry choices are gameplay choices, not art choices.** Moving a room's center to make an isometric floor plan look better changes duel ranges and therefore changes who wins — which is why any geometry edit drifts the golden-file fixture and has to be a deliberate, re-blessed decision, and why a dedicated pacing report (not eyeballing) gates rotation-timing changes.
- **Multi-season play reveals different failure modes than one match does.** A league-balance overhaul (tracked via [scripts/snowball_report.py](#)) was needed after headless multi-season runs showed condition (form/ morale) snowballing into repeated 13–0/13–1 blowouts that single-match testing never surfaced.

8. Roadmap (see [ROADMAP.md](#) for the living version)

North-star bets the whole project is aimed at:

1. **Matches feel alive** — different seeds, same rosters, visibly different legible stories. (Substantially proven out; ongoing tuning.)
2. **An LLM can competently play a full season** through the headless state/action API — the same contract the web UI and any future RL agent use. (API shape exists; a dedicated playtest harness is next.)
3. **A world model can sample a plausible season** from event-sequence data — the research capstone, gated on the first two bets holding up over real logged play.

Shipped so far (see [ROADMAP.md](#) changelog for exact commits): the full match engine with the fallback/retake model; the full management loop across multiple seasons; the web app and isometric viewer; floor geometry with props, elevation, and continuous movement; micro-combat (peeks, flanks, footwork); rotation pacing tuned to a real-feel ~30s rule; season analytics and grounded narrative; scouting fog, map veto, staff, sponsorships, and the Talk module.

In flight right now: map gimmicks (rotating doors, teleporters, breakable doors) are implemented end-to-end in the schema and engine (noise propagation, watch-direction reactions, closed-door state) but not yet authored onto Lotus, Bind, or Ascent's actual map data — that content pass, plus re-blessing the golden fixture once the engine change is final, is the immediate next step.

Next candidates (unscheduled, in rough order of how they were left): site-geometry passes on Ascent/Bind to pull their attack rate closer to the other three maps' band; camera follow/zoom in the viewer; a headless LLM-playtest harness (north-star bet #2); an RL [env](#) wrapper over the same headless contract (Track B); deepening personality/relationship systems beyond the current tag-based model (Track A); and, eventually, the event-sequence world-model research arm (Track C). None of this is committed scope until it lands on the roadmap's Now/Next list — this section is a map of the terrain, not a promise.

9. Explicit non-goals

- Real Valorant pro names, real statistical profiles, or Riot Games assets (a fictional roster and original agent kits serve the same design goals legally and creatively).
- True 3D rendering — the isometric floor-plan viewer is the intentional ceiling; it gets most of the tactical legibility of 3D without the engine-building cost.
- Multiplayer/network play, a mobile port, a commercial Steam release, real-time voice comms, betting/fantasy mechanics, or scraping real Valorant API/Riot data. None of these serve the north-star bets above.